

Bài 8. Phân tích hồi qui

Tìm hiểu xong bài này bạn sẽ:

- Hiểu được một số mô hình phân tích hồi qui: hồi qui tuyến tính một biến, hồi qui tuyến tính đa biến, hồi qui logistic
- Sử dụng được thư viện sklearn để xây dựng các mô hình hồi qui

Nội dung

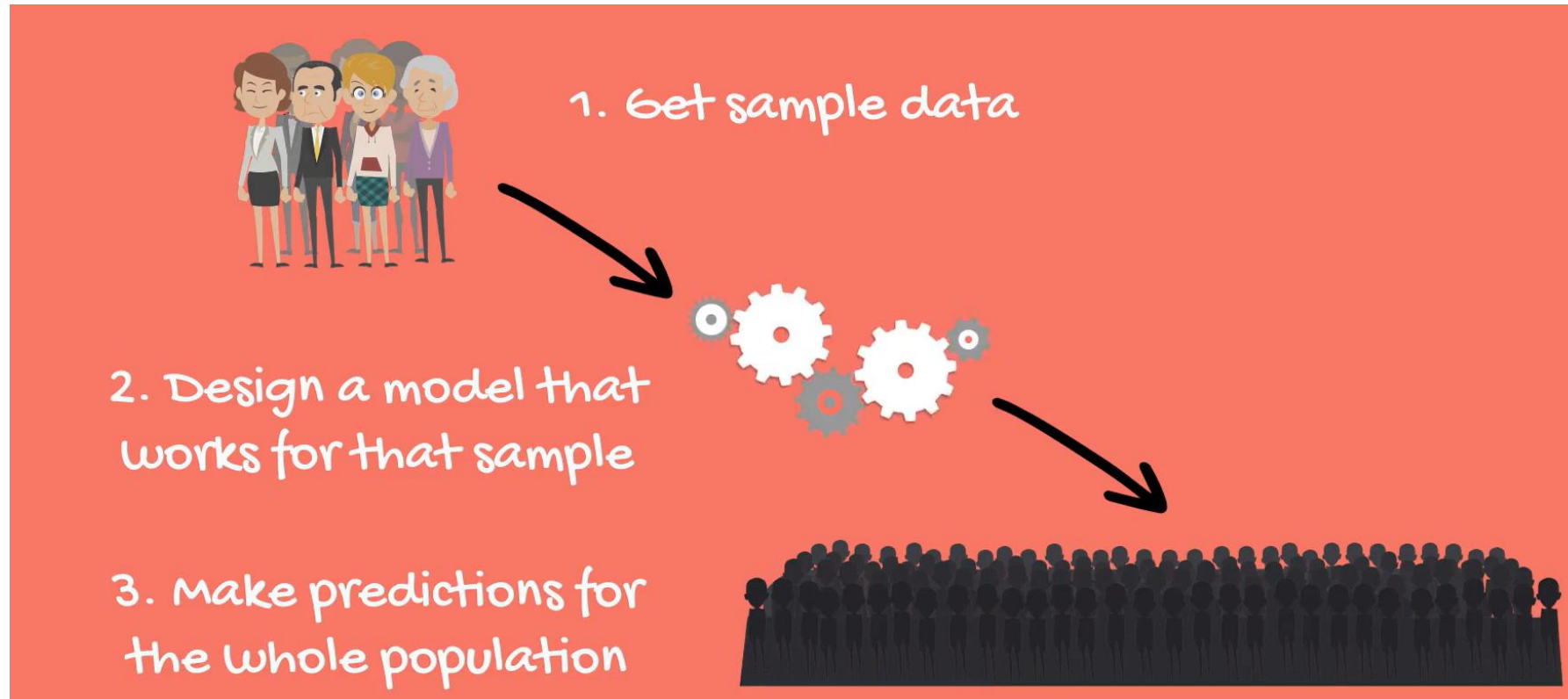
1. Hồi qui tuyến tính một biến
2. Hồi qui tuyến tính đa biến
3. Hồi qui tuyến tính với sklearn
4. Hồi qui logistic

1. Simple linear regression

What is linear regression?

- Phân tích hồi quy là một trong những phương pháp được sử dụng rộng rãi nhất để dự đoán. Hồi quy tuyến tính có lẽ là một trong những phương pháp học máy cơ bản nhất và là điểm khởi đầu cho con đường học tập phân tích tiên tiến của mỗi nhà khoa học dữ liệu đầy tham vọng.
- Một hồi quy tuyến tính là một sự xấp xỉ tuyến tính của mối quan hệ nhân quả giữa hai hoặc nhiều biến.
- Các mô hình hồi quy rất có giá trị, vì chúng là một trong những cách phổ biến nhất để suy luận và dự đoán. Ngoài ra, phân tích hồi quy cũng được sử dụng để xác định và đánh giá các yếu tố ảnh hưởng đến một kết quả cụ thể một cách có ý nghĩa.
- Giống như nhiều kỹ thuật thống kê khác, các mô hình hồi quy giúp chúng ta đưa ra dự đoán về tổng thể dựa trên dữ liệu mẫu

Process



Model



DEPENDENT
/predicted/

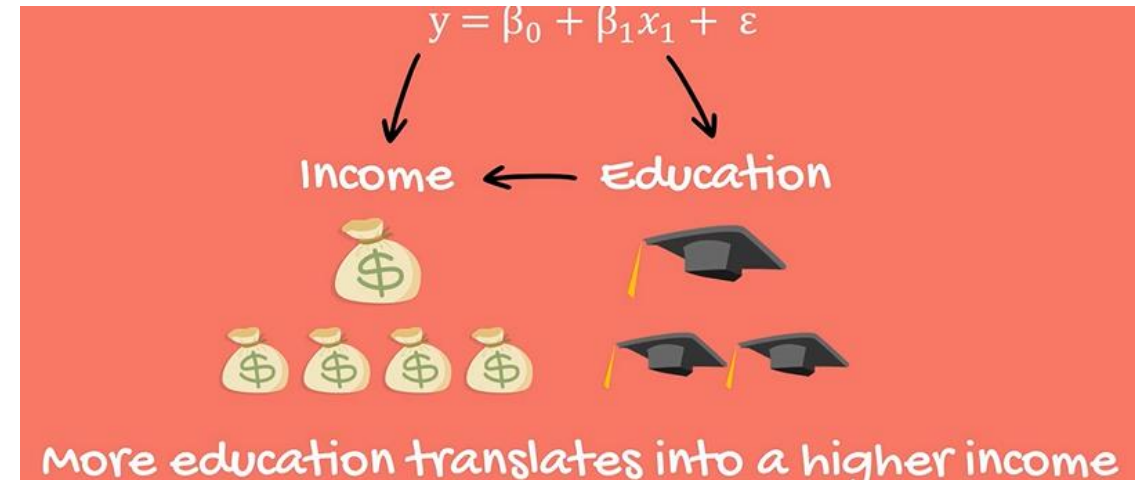
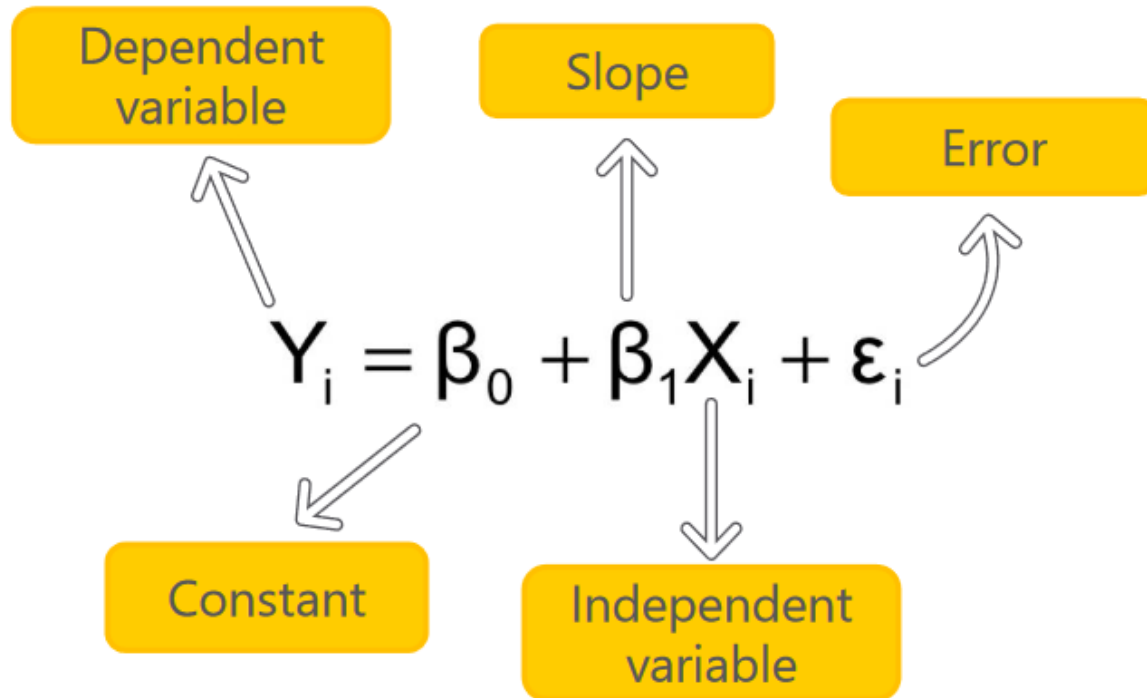
INDEPENDENT
/predictors/



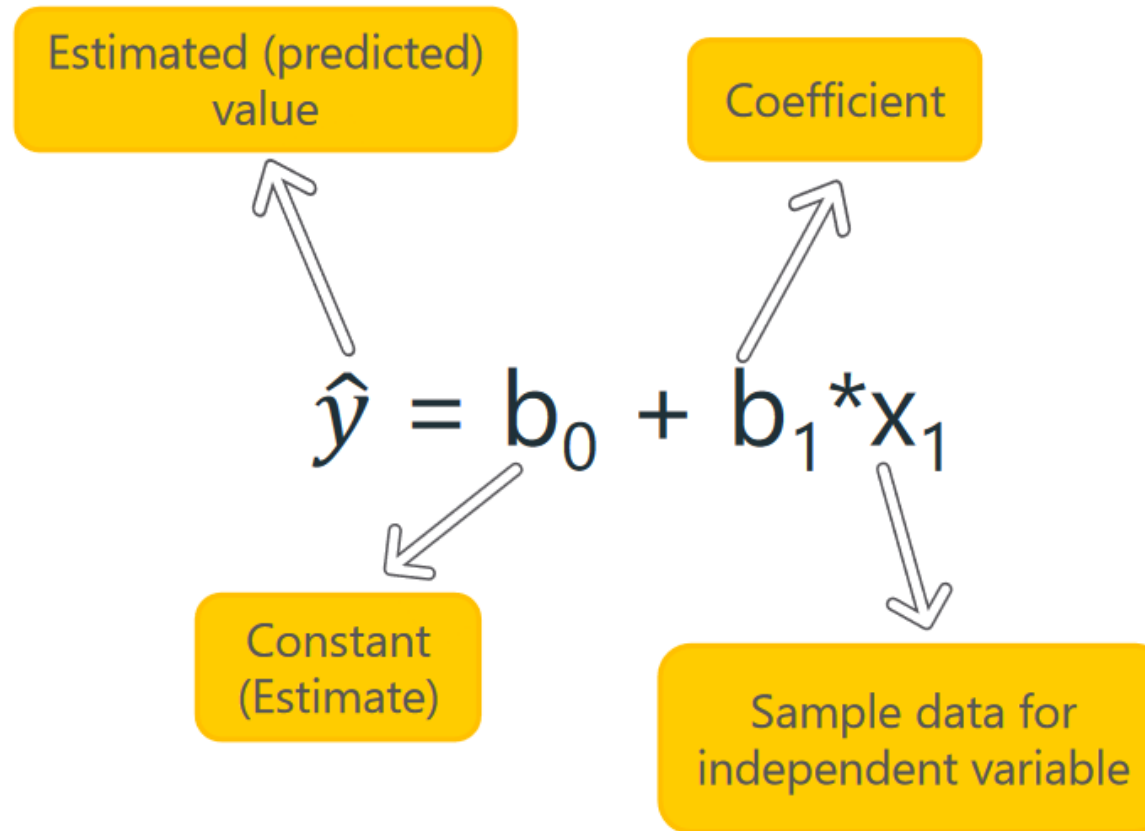
$$Y = F(x_1, x_2, \dots, x_k)$$

The dependent variable Y is a function of the independent variables x_1 to x_k

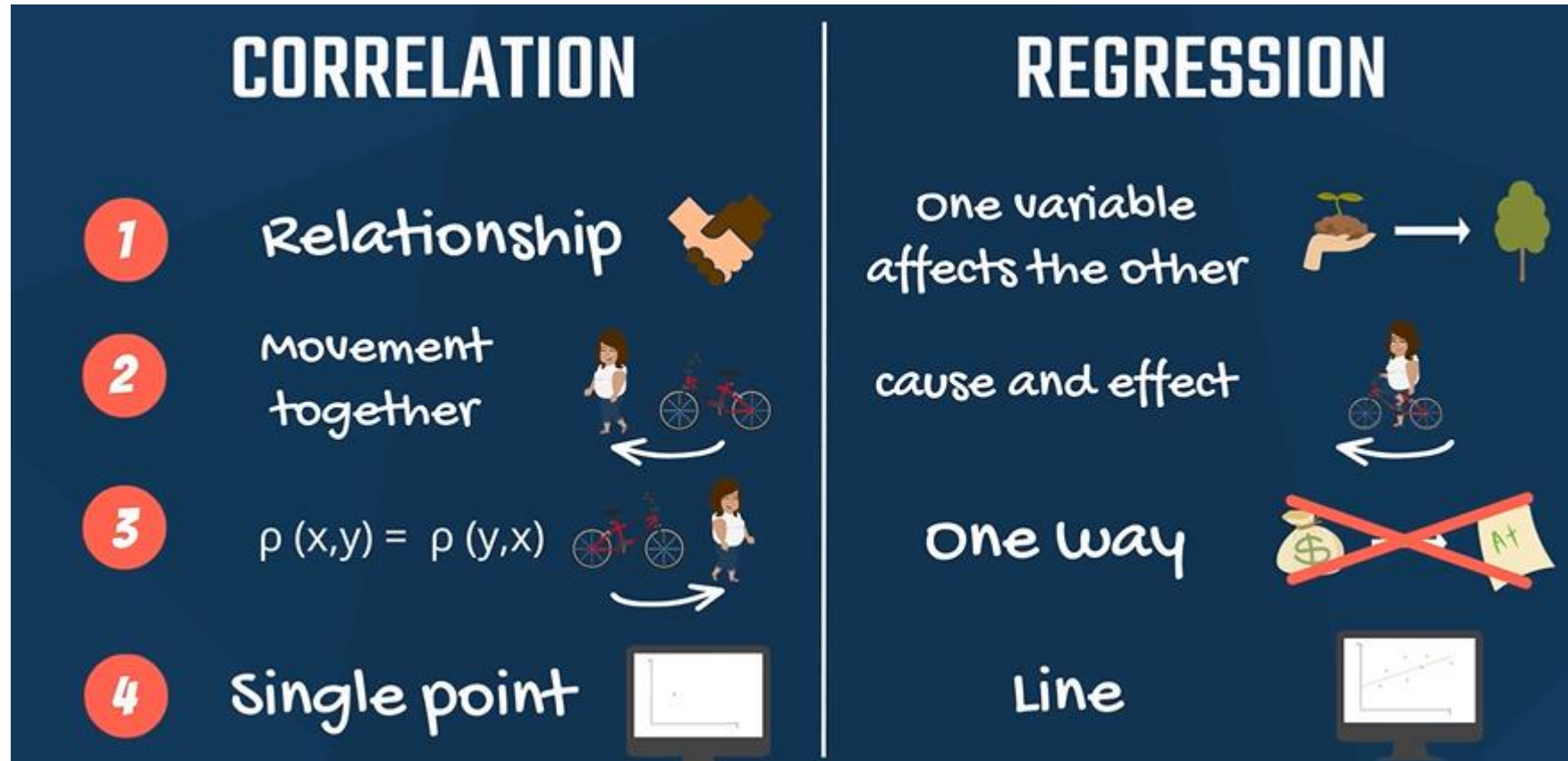
The Simple regression model



The linear regression equation



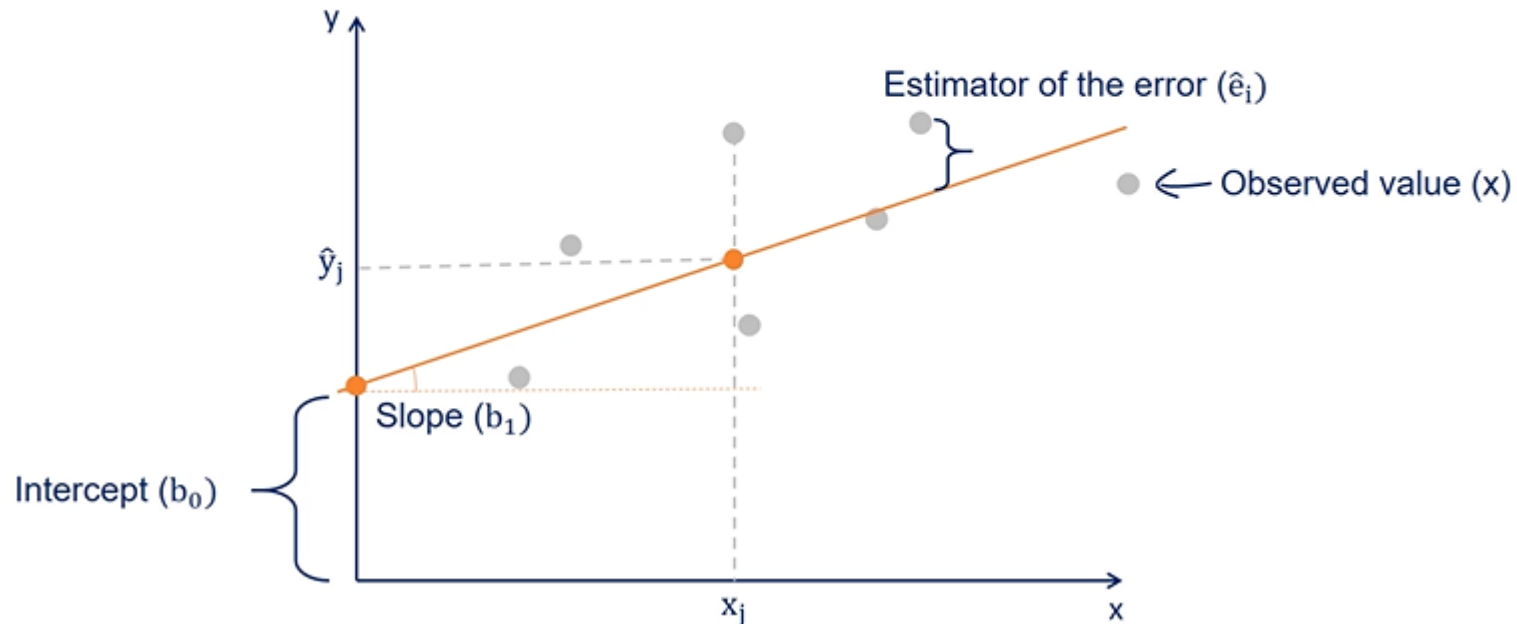
Correlation vs Regression



Geometrical representation of linear regression



$$\hat{y}_i = b_0 + b_1 x_i$$



Example: Linear Regression with statsmodel



- Xây dựng mô hình hồi qui giữa điểm trung bình(GPA) và kết quả bài thi (SAT)

```
[3]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import statsmodels.api as sm
```

```
[4]: df = pd.read_csv("D:/0.LearningResources/2.DataScience/2.Data/simple_linear_regression.csv")
```

```
[5]: df.head()
```

```
[5]:
```

	SAT	GPA
0	1714	2.40
1	1664	2.52
2	1760	2.54
3	1685	2.74
4	1693	2.83

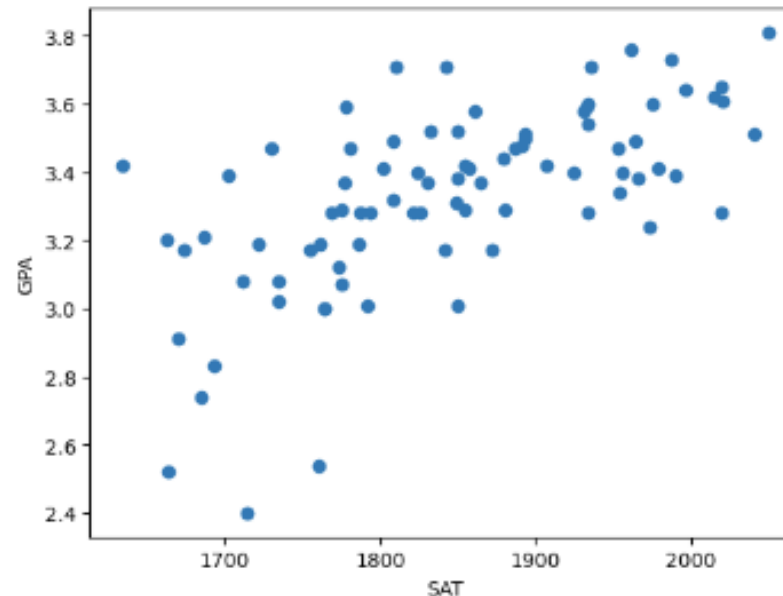
Example: Linear Regression with statsmodel



- Xây dựng mô hình hồi qui giữa điểm trung bình (GPA) và kết quả bài thi (SAT)

```
y = df['GPA']  
x1 = df['SAT']
```

```
plt.scatter(x1, y)  
plt.xlabel('SAT')  
plt.ylabel('GPA')  
plt.show()
```



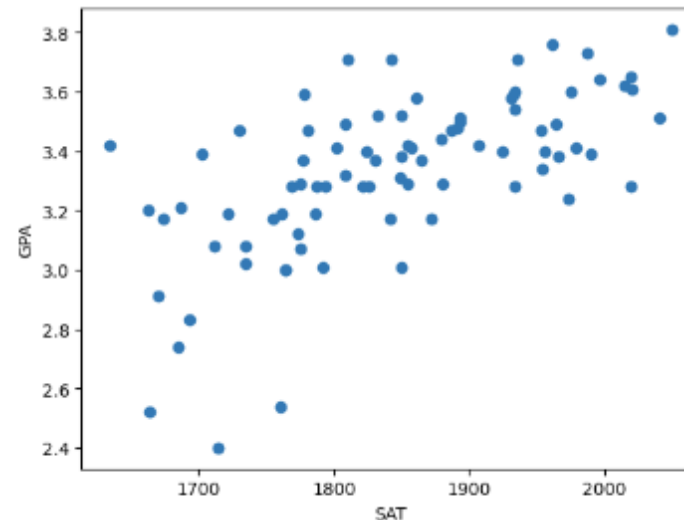
Example: Linear Regression with statsmodel



- Xây dựng mô hình hồi qui giữa điểm trung bình(GPA) và kết quả bài thi (SAT)

```
y = df['GPA']  
x1 = df['SAT']
```

```
plt.scatter(x1, y)  
plt.xlabel('SAT')  
plt.ylabel('GPA')  
plt.show()
```



Example: Linear Regression with statsmodel



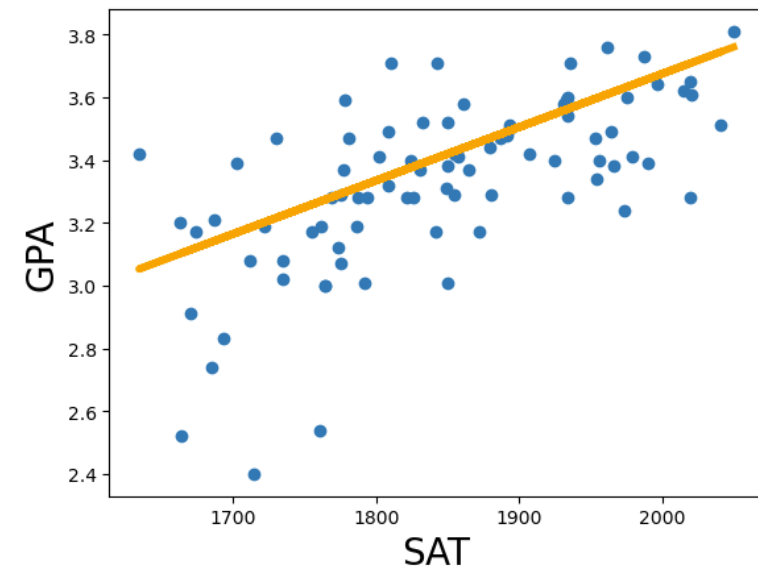
- Xây dựng mô hình hồi qui giữa điểm trung bình(GPA) và kết quả bài thi (SAT)

```
x = sm.add_constant(x1)
results = sm.OLS(y, x).fit()
results.summary()
```

OLS Regression Results

Dep. Variable:	GPA	R-squared:	0.406			
Model:	OLS	Adj. R-squared:	0.399			
Method:	Least Squares	F-statistic:	56.05			
Date:	Fri, 19 Apr 2024	Prob (F-statistic):	7.20e-11			
Time:	22:13:58	Log-Likelihood:	12.672			
No. Observations:	84	AIC:	-21.34			
Df Residuals:	82	BIC:	-16.48			
Df Model:	1					
Covariance Type:	nonrobust					
	coef	std err	t	P> t	[0.025	0.975]
const	0.2750	0.409	0.673	0.503	-0.538	1.088
SAT	0.0017	0.000	7.487	0.000	0.001	0.002

```
plt.scatter(x1, y)
yhat = 0.0017*x1 + 0.275
fig = plt.plot(x1, yhat, lw=4, c='orange', label='regression line')
plt.xlabel('SAT', fontsize=20)
plt.ylabel('GPA', fontsize=20)
plt.show()
```



How to interpret the regression table



Out[7]:

OLS Regression Results

Dep. Variable:	GPA	R-squared:	0.406
Model:	OLS	Adj. R-squared:	0.399
Method:	Least Squares	F-statistic:	56.05
Date:	Thu, 03 May 2018	Prob (F-statistic):	7.20e-11
Time:	14:24:59	Log-Likelihood:	12.672
No. Observations:	84	AIC:	-21.34
Df Residuals:	82	BIC:	-16.48
Df Model:	1		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
const	0.2750	0.409	0.673	0.503	-0.538	1.088
SAT	0.0017	0.000	7.487	0.000	0.001	0.002
Omnibus:	12.839	Durbin-Watson:	0.950			
Prob(Omnibus):	0.002	Jarque-Bera (JB):	16.155			
Skew:	-0.722	Prob(JB):	0.000310			
Kurtosis:	4.590	Cond. No.	3.29e+04			



How to interpret the regression table



	coef	std err	t	P> t	[0.025	0.975]
const	0.2750	0.409	0.673	0.503	-0.538	1.088
SAT	0.0017	0.000	7.487	0.000	0.001	0.002

$$\hat{y} = b_0 + b_1 x_1$$

↓
0.275

$$\hat{y} = 0.275 + 0.0017x_1$$
$$\text{GPA} = 0.275 + 0.0017 * \text{SAT}$$

	coef	std err	t	P> t	[0.025	0.975]
const	0.2750	0.409	0.673	0.503	-0.538	1.088
SAT	0.0017	0.000	7.487	0.000	0.001	0.002

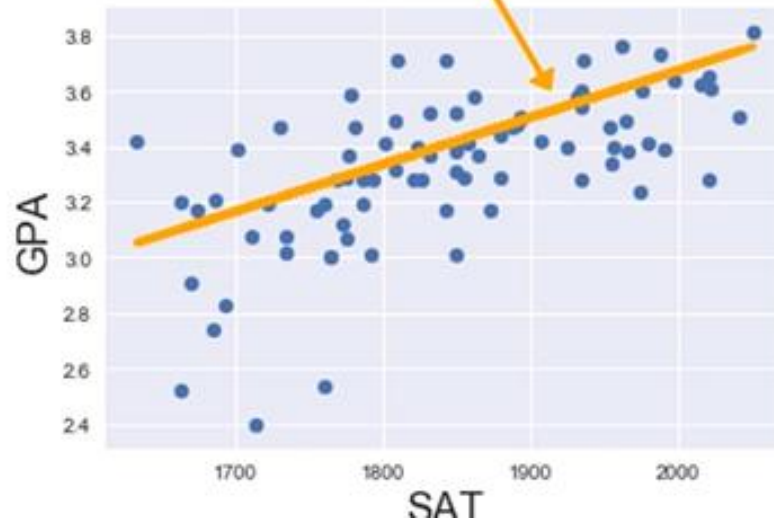
$$\hat{y} = b_0 + b_1 x_1$$

↓ ↓
0.275 0.0017

How to interpret the regression table

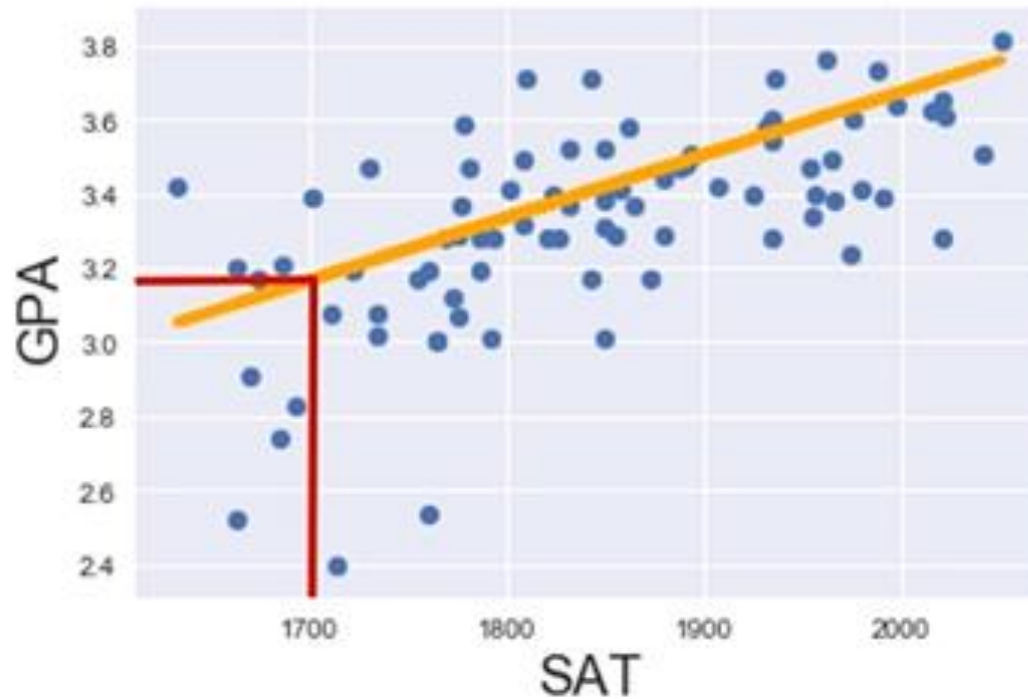


```
8]: plt.scatter(x1,y)
    yhat = 0.0017*x1 + 0.275
    fig = plt.plot(x1,yhat, lw=4, c='orange', label='regression line')
    plt.xlabel('SAT', fontsize = 20)
    plt.ylabel('GPA', fontsize = 20)
    plt.show()
```



$$\hat{y} = 0.275 + 0.0017x_1$$
$$\text{GPA} = 0.275 + 0.0017 * \text{SAT}$$

How to interpret the regression table



$$\hat{y} = 0.275 + 0.0017x_1$$

$$\text{GPA} = 0.275 + 0.0017 * \text{SAT}$$

$$3.165 = 0.275 + 0.0017 * 1700$$

How to interpret the regression table



Covariance Type: nonrobust

	coef	std err	t	<u>P> t </u>	[0.025	0.975]
const	0.2750	0.409	0.673	0.503	-0.538	1.088
SAT	0.0017	0.000	7.487	0.000	0.001	0.002

Omnibus: 12.839 Durbin-Watson: 0.950
Prob(Omnibus): 0.002 Jarque-Bera (JB): 16.155



A p-value < 0.05, means that the variable is significant

	coef	std err	t	<u>P> t </u>	[0.025	0.975]
const	0.2750	0.409	0.673	0.503	-0.538	1.088
SAT	0.0017	0.000	7.487	<u>0.000</u>	0.001	0.002

Omnibus: 12.839 Durbin-Watson: 0.950
Prob(Omnibus): 0.002 Jarque-Bera (JB): 16.155



SAT is a significant variable when predicting college GPA

Decomposition of variability

SST
TSS

SUM OF SQUARES TOTAL

Measures the total variability of the dataset

$$\sum_{i=1}^n (y_i - \bar{y})^2$$


SSR
ESS

SUM OF SQUARES REGRESSION

Measures the explained variability by your line

$$\sum_{i=1}^n (\hat{y}_i - \bar{y})^2$$


SSE
RSS

SUM OF SQUARES ERROR

Measures the unexplained variability by the regression

$$\sum_{i=1}^n e_i^2$$


Decomposition of variability

$$SST = SSR + SSE$$

CONNECTION?

Total variability = Explained variability + Unexplained variability

$$\sum_{i=1}^n (y_i - \bar{y})^2 = \sum_{i=1}^n (\hat{y}_i - \bar{y})^2 + \sum_{i=1}^n e_i^2$$



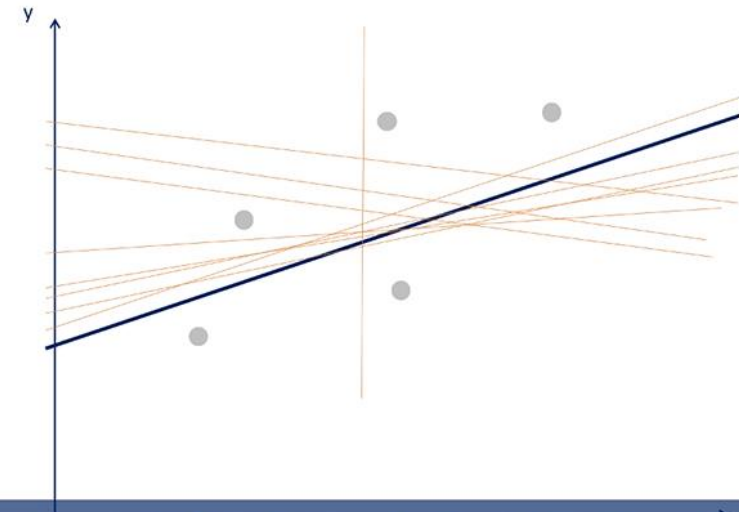
What is OLS model?

Ordinary least squares

↓
 $\min SSE$

Lower error $\Rightarrow$ better explanatory power

LOWEST error $\Rightarrow$ BEST explanatory power



Graphically the regression line is the one closest to all points simultaneously

R-squared?

OLS Regression Results

Dep. Variable:	GPA	R-squared:	0.406
Model:	OLS	Adj. R-squared:	0.399
Method:	Least Squares	F-statistic:	56.05
Date:	Thu, 03 May 2018	Prob (F-statistic):	7.20e-11
Time:	14:24:59	Log-Likelihood:	12.672
No. Observations:	84	AIC:	-21.34
Df Residuals:	82	BIC:	-16.48
Df Model:	1		
Covariance Type:	nonrobust		

R^2

0

1

Your regression
explains **NONE** of
the variability

R^2

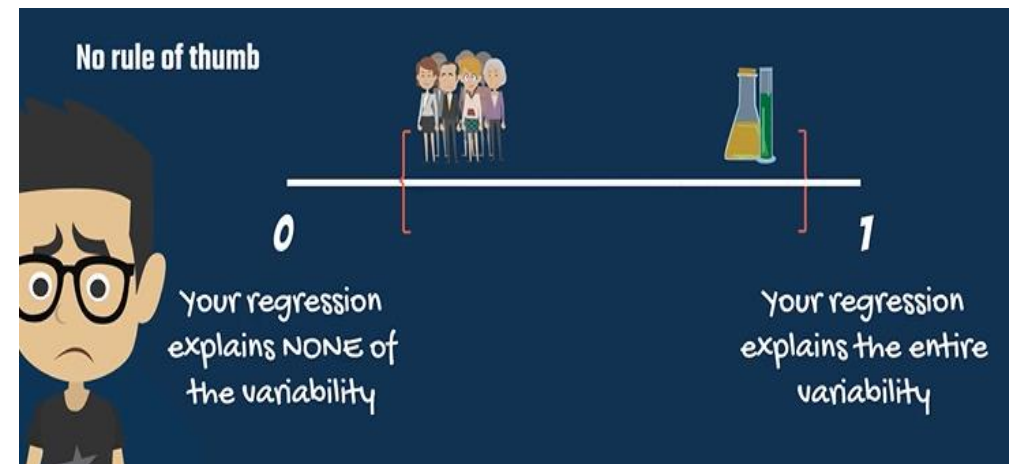
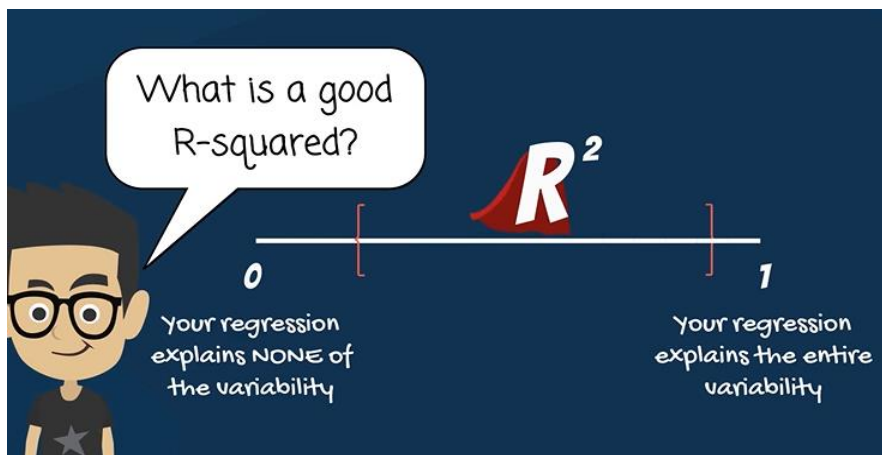
=

variability explained
by the regression

$$\frac{SSR}{SST}$$

Total variability of
the dataset

R-squared?



R-squared?

$$R^2 = 0.406$$

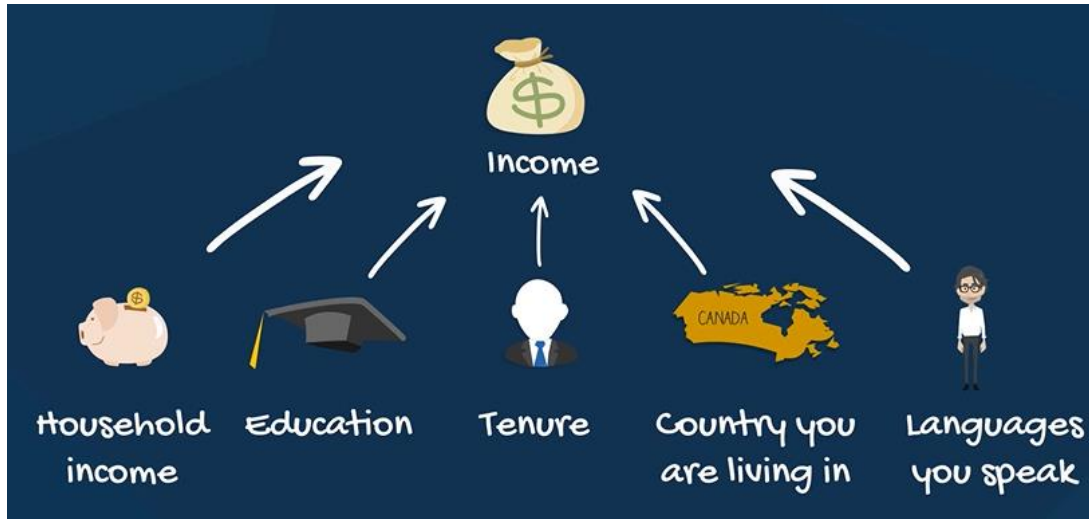
$$\text{College GPA} = 0.275 + 0.0017 * \text{SAT}$$

Out[7]: OLS Regression Results

Dep. Variable:	GPA	R-squared:	0.406
Model:	OLS	Adj. R-squared:	0.399
Method:	Least Squares	F-statistic:	56.05
Date:	Thu, 03 May 2018	Prob (F-statistic):	7.20e-11
Time:	14:24:59	Log-Likelihood:	12.672
No. Observations:	84	AIC:	-21.34
Df Residuals:	82	BIC:	-16.48
Df Model:	1		
Covariance Type:	nonrobust		

2. Multiple Linear Regression

Multiple Linear Regression



Good models require multiple regressions, in order to address the higher complexity of problems



Multiple Linear Regression Model

$$\gamma = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_k x_k + \varepsilon$$

Multiple regression equation

$$\hat{y} = b_0 + b_1x_1 + b_2x_2 + \cdots + b_kx_k$$

After 3 dimensions, there is no visual way to represent the data

min **SSE**



IT'S ABOUT THE BEST FITTING MODEL

Optimal number of variable to use

Adjusted R-squared



1. The R-squared measures how much of the total variability is explained by our model
2. Multiple regressions are always better than simple ones, as with each additional variable you add, the explanatory power may only increase or stay the same

$$\bar{R}^2 < R^2$$

The adjusted R-squared penalizes excessive use of variables

Example

	SAT	GPA	Rand 1,2,3
count	84.000000	84.000000	84.000000
mean	1845.273810	3.330238	2.059524
std	104.530661	0.271617	0.855192
min	1634.000000	2.400000	1.000000
25%	1772.000000	3.190000	1.000000
50%	1846.000000	3.380000	2.000000
75%	1934.000000	3.502500	3.000000
gle output scrolling	000	3.810000	3.000000

$$\text{GPA} = b_0 + b_1\text{SAT} + b_2\text{Rand1,2,3}$$

Example

Out[8]:

OLS Regression Results

Dep. Variable:	GPA	R-squared:	0.407
Model:	OLS	Adj. R-squared:	0.392
Method:	Least Squares	F-statistic:	27.76
Date:	Thu, 10 May 2018	Prob (F-statistic):	6.58e-10
Time:	14:59:06	Log-Likelihood:	12.720
No. Observations:	84	AIC:	-19.44
Df Residuals:	81	BIC:	-12.15
Df Model:	2		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
const	0.2960	0.417	0.710	0.480	-0.533	1.125
SAT	0.0017	0.000	7.432	0.000	0.001	0.002
Rand 1,2,3	-0.0083	0.027	-0.304	0.762	-0.062	0.046

Omnibus:	12.992	Durbin-Watson:	0.948
Prob(Omnibus):	0.002	Jarque-Bera (JB):	16.364
Skew:	-0.731	Prob(JB):	0.000280
Kurtosis:	4.594	Cond. No.	3.33e+04

Old tables

Dep. Variable:	GPA	R-squared:	0.406
Model:	OLS	Adj. R-squared:	0.399
Method:	Least Squares	F-statistic:	56.05
Date:	Thu, 05 Apr 2018	Prob (F-statistic):	7.20e-11
Time:	15:24:09	Log-Likelihood:	12.672
No. Observations:	84	AIC:	-21.34
Df Residuals:	82	BIC:	-16.48
Df Model:	1		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
const	0.2750	0.409	0.673	0.503	-0.538	1.088
SAT	0.0017	0.000	7.487	0.000	0.001	0.002

Omnibus:	12.839	Durbin-Watson:	0.950
Prob(Omnibus):	0.002	Jarque-Bera (JB):	16.155
Skew:	-0.722	Prob(JB):	0.000310
Kurtosis:	4.590	Cond. No.	3.29e+04

Date: Thu, 10 May 2018 Prob (F-statistic): 6.58

$$H_0: \beta = 0$$

Example

In [8]: `results.summary()`

Out[8]: OLS Regression Results

Dep. Variable:	GPA	R-squared:	0.407
Model:	OLS	Adj. R-squared:	0.392
Method:	Least Squares	F-statistic:	27.76
Date:	Thu, 10 May 2018	Prob (F-statistic):	6.58e-10
Time:	14:59:06	Log-Likelihood:	12.720
No. Observations:	84	AIC:	-19.44
Df Residuals:	81	BIC:	-12.15
Df Model:	2		

~~$\hat{y}_2 = 0.296 + 0.0017x_1 - 0.0083x_2$~~

	coef	std err	t	P> t	[0.025	0.975]
const	0.2960	0.417	0.710	0.480	-0.533	1.125
SAT	0.0017	0.000	7.432	0.000	0.001	0.002
Rand 1,2,3	-0.0083	0.027	-0.304	0.762	-0.062	0.046

Omnibus:	12.992	Durbin-Watson:	0.948
Prob(Omnibus):	0.002	Jarque-Bera (JB):	16.364
Skew:	-0.731	Prob(JB):	0.000280
Kurtosis:	4.594	Cond. No.	3.33e+04

Old tables

Dep. Variable:	GPA	R-squared:	0.406
Model:	OLS	Adj. R-squared:	0.399
Method:	Least Squares	F-statistic:	56.05
Date:	Thu, 05 Apr 2018	Prob (F-statistic):	7.20e-11
Time:	15:24:09	Log-Likelihood:	12.672
No. Observations:	84	AIC:	-21.34
Df Residuals:	82	BIC:	-16.48

$\hat{y}_1 = 0.275 + 0.0017x_1$

	coef	std err	t	P> t	[0.025	0.975]
const	0.2750	0.409	0.673	0.503	-0.538	1.088
SAT	0.0017	0.000	7.487	0.000	0.001	0.002

Omnibus:	12.839	Durbin-Watson:	0.950
Prob(Omnibus):	0.002	Jarque-Bera (JB):	16.155
Skew:	-0.722	Prob(JB):	0.000310
Kurtosis:	4.590	Cond. No.	3.29e+04

Test for significance of the model (F-test)



F-test:

$$H_0: \beta_1 = \beta_2 = \dots = \beta_k = 0$$

H_1 : at least one $\beta_i \neq 0$

Out[7]: OLS Regression Results

Dep. Variable:	GPA	R-squared:	0.406
Model:	OLS	Adj. R-squared:	0.399
Method:	Least Squares	F-statistic:	56.05
Date:	Thu, 03 May 2018	Prob (F-statistic):	7.20e-11
Time:	14:24:59	Log-Likelihood:	12.672
No. Observations:	84	AIC:	-21.34
Df Residuals:	82	BIC:	-16.48
Df Model:	1		
Covariance Type:	nonrobust		

GPA = F (SAT, Rand 123)

Dep. Variable:	GPA	R-squared:	0.407
Model:	OLS	Adj. R-squared:	0.392
Method:	Least Squares	F-statistic:	27.76
Date:	Tue, 06 Mar 2018	Prob (F-statistic):	6.58e-10 - 0.000
Time:	18:09:00	Log-Likelihood:	12.720
No. Observations:	84	AIC:	-19.44
Df Residuals:	81	BIC:	-12.15
Df Model:	2		
Covariance Type:	nonrobust		

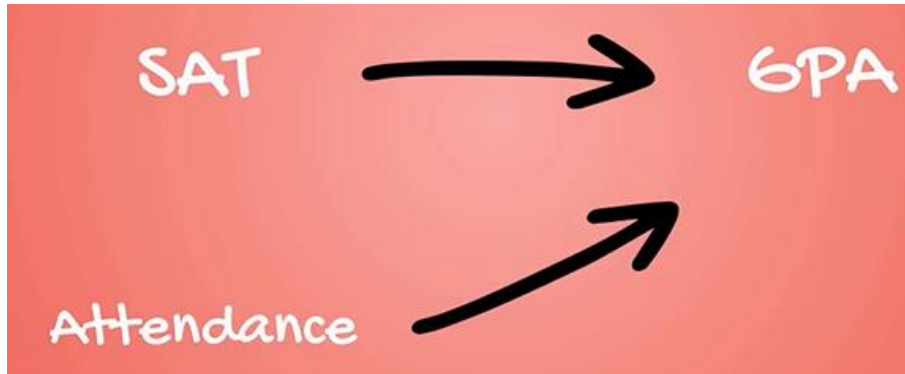
The lower the F-statistic, the closer to a non-significant model

OLS Assumptions

1. Linearity $y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_k x_k + \varepsilon$
2. No endogeneity $\sigma_{x\varepsilon} = 0 : \forall x, \varepsilon$
3. Normality and homoscedasticity $\varepsilon \sim N(0, \sigma^2)$
4. No autocorrelation $\sigma_{\varepsilon_i \varepsilon_j} = 0 : \forall i \neq j$
5. No multicollinearity $\rho_{x_i x_j} \neq 1 : \forall i, j; i \neq j$

The biggest mistake you can make is to perform a regression that violates one of these assumptions!

Dealing with Categorical data - Dummy Variables



This is categorical data, so we cannot simply put it in the regression

Dummy -> imitation of categories with numbers!

SAT	GPA	Attendance
1714	2.4	No
1664	2.52	No
1760	2.54	No
1685	2.74	No
1693	2.83	No
1670	2.91	No
1764	3	No
1764	3	No
1792	3.01	No
1850	3.01	No
1735	3.02	No
1775	3.07	No
1735	3.08	No

Yes = 1

No = 0

Dealing with Categorical data - Dummy Variables



```
data['Attendance'] = data['Attendance'].map({'Yes':1,'No': 0})
```

```
y = data['GPA']  
x1 = data[['SAT','Attendance']]
```

```
x = sm.add_constant(x1)  
results = sm.OLS(y,x).fit()  
results.summary()
```

Dep. Variable:	GPA	R-squared:	0.565
Model:	OLS	Adj. R-squared:	0.555
Method:	Least Squares	F-statistic:	52.70
Date:	Fri, 11 May 2018	Prob (F-statistic):	2.19e-15
Time:	16:08:28	Log-Likelihood:	25.798
No. Observations:	84	AIC:	-45.60
Df Residuals:	81	BIC:	-38.30
Df Model:	2		

	coef	std err	t	P> t	[0.025	0.975]
const	0.6439	0.358	1.797	0.076	-0.069	1.357
SAT	0.0014	0.000	7.141	0.000	0.001	0.002
Attendance	0.2226	0.041	5.451	0.000	0.141	0.304

Dealing with Categorical data - Dummy Variables



OLS Regression Results

Dep. Variable:	GPA	R-squared:	0.565
Model:	OLS	Adj. R-squared:	0.555
Method:	Least Squares	F-statistic:	52.70
Date:	Fri, 11 May 2018	Prob (F-statistic):	2.19e-15
Time:	16:08:28	Log-Likelihood:	25.798
No. Observations:	84	AIC:	-45.60
Df Residuals:	81	BIC:	-38.30
Df Model:	2		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
const	0.6439	0.358	1.797	0.076	-0.069	1.357
SAT	0.0014	0.000	7.141	0.000	0.001	0.002
Attendance	0.2226	0.041	5.451	0.000	0.141	0.304

Omnibus:	19.560	Durbin-Watson:	1.009
Prob(Omnibus):	0.000	Jarque-Bera (JB):	27.189

Dep. Variable:	GPA	R-squared:	0.406
Model:	OLS	Adj. R-squared:	0.399
Method:	Least Squares	F-statistic:	56.05
Date:	Thu, 05 Apr 2018	Prob (F-statistic):	7.20e-11
Time:	15:24:09	Log-Likelihood:	12.672
No. Observations:	84	AIC:	-21.34
Df Residuals:	82	BIC:	-16.48
Df Model:	1		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
const	0.2750	0.409	0.673	0.503	-0.538	1.088
SAT	0.0017	0.000	7.487	0.000	0.001	0.002

Omnibus:	12.839	Durbin-Watson:	0.950
Prob(Omnibus):	0.002	Jarque-Bera (JB):	16.155
Skew:	-0.722	Prob(JB):	0.000310
Kurtosis:	4.590	Cond. No.	3.29e+04

Dealing with Categorical data - Dummy Variables



Dep. Variable:	GPA	R-squared:	0.565
Model:	OLS	Adj. R-squared:	0.555
Method:	Least Squares	F-statistic:	52.70
Date:	Fri, 11 May 2018	Prob (F-statistic):	2.19e-15
Time:	16:08:28	Log-Likelihood:	25.798
No. Observations:	84	AIC:	-45.60
Df Residuals:	81	BIC:	-38.30

$$\text{GPA} = 0.6439 + 0.0014 * \text{SAT} + 0.2226 * \text{Dummy}$$

	coef	std err	t	P> t	[0.025	0.975]
const	0.6439	0.358	1.797	0.076	-0.069	1.357
SAT	0.0014	0.000	7.141	0.000	0.001	0.002
Attendance	0.2226	0.041	5.451	0.000	0.141	0.304

Dep. Variable:	GPA	R-squared:	0.406
Model:	OLS	Adj. R-squared:	0.399
Method:	Least Squares	F-statistic:	56.05
Date:	Thu, 05 Apr 2018	Prob (F-statistic):	7.20e-11
Time:	15:24:09	Log-Likelihood:	12.672
No. Observations:	84	AIC:	-21.34
Df Residuals:	82	BIC:	-16.48

$$\text{GPA} = 0.275 + 0.0017 * \text{SAT}$$

	coef	std err	t	P> t	[0.025	0.975]
const	0.2750	0.409	0.673	0.503	-0.538	1.088
SAT	0.0017	0.000	7.487	0.000	0.001	0.002

Omnibus: 12.839 Durbin-Watson: 0.950

Attended (dummy = 1)

$$\text{GPA} = 0.8665 + 0.0014 * \text{SAT}$$

$$\text{GPA} = 0.6439 + 0.0014 * \text{SAT} + 0.2226 * 1$$

$$\text{GPA} = 0.6439 + 0.0014 * \text{SAT} + 0.2226 * \text{Dummy}$$

$$\text{GPA} = 0.6439 + 0.0014 * \text{SAT} + 0.2226 * 0$$

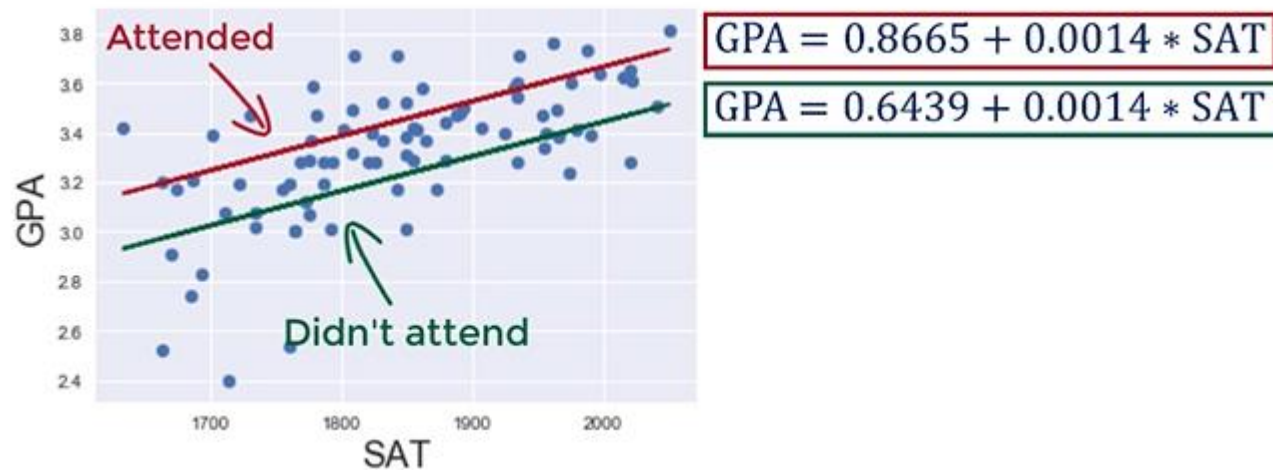
$$\text{GPA} = 0.6439 + 0.0014 * \text{SAT}$$

Did not attend (dummy = 0)

Dealing with Categorical data - Dummy Variables



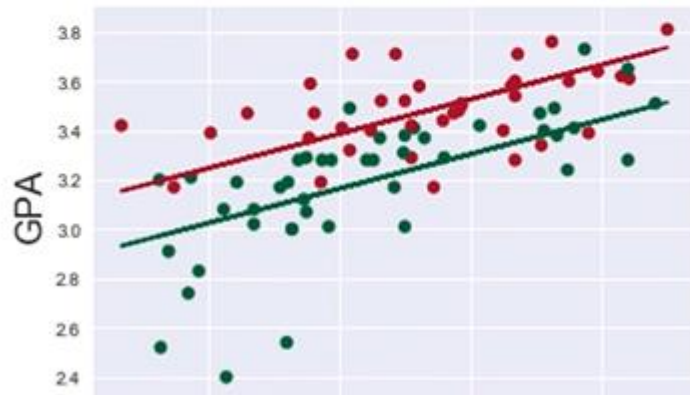
```
In [11]: plt.scatter(data['SAT'],y)
yhat_no = 0.6439 + 0.0014*data['SAT']
yhat_yes = 0.8665 + 0.0014*data['SAT']
fig = plt.plot(data['SAT'],yhat_no, lw=2, c='#006837')
fig = plt.plot(data['SAT'],yhat_yes, lw=2, c='#a50026')
plt.xlabel('SAT', fontsize = 20)
plt.ylabel('GPA', fontsize = 20)
plt.show()
```



Dealing with Categorical data - Dummy Variables



```
: plt.scatter(data['SAT'],y,c=data['Attendance'],cmap='RdYlGn_r')
yhat_no = 0.6439 + 0.0014*data['SAT']
yhat_yes = 0.8665 + 0.0014*data['SAT']
fig = plt.plot(data['SAT'],yhat_no, lw=2, c='#006837')
fig = plt.plot(data['SAT'],yhat_yes, lw=2, c='#a50026')
plt.xlabel('SAT', fontsize = 20)
plt.ylabel('GPA', fontsize = 20)
plt.show()
```



Make prediction

Bob, who got 1700 on the SAT and did NOT attend

Alice, who got 1670 on the SAT and attended

```
5]: new_data = pd.DataFrame({'const':1, 'SAT':[1700,1670], 'Attendance':[0,1]})
new_data = new_data[['const', 'SAT', 'Attendance']]
new_data
```

5]:

	const	SAT	Attendance
0	1	1700	0
1	1	1670	1

```
predictions = results.predict(new_data)
predictions
```

```
0    3.023513
1    3.204163
dtype: float64
```

```
In [18]: predictionsdf=pd.DataFrame({'Predictions':predictions})
joined = new_data.join(predictionsdf)
joined.rename(index={0:'Bob',1:'Alice'})
```

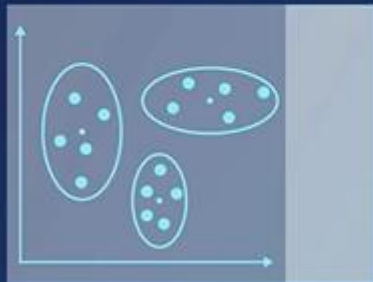
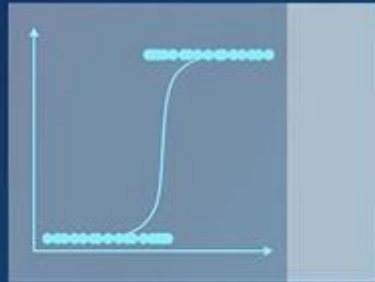
Out[18]:

	const	SAT	Attendance	Predictions
Bob	1	1700	0	3.023513
Alice	1	1670	1	3.204163

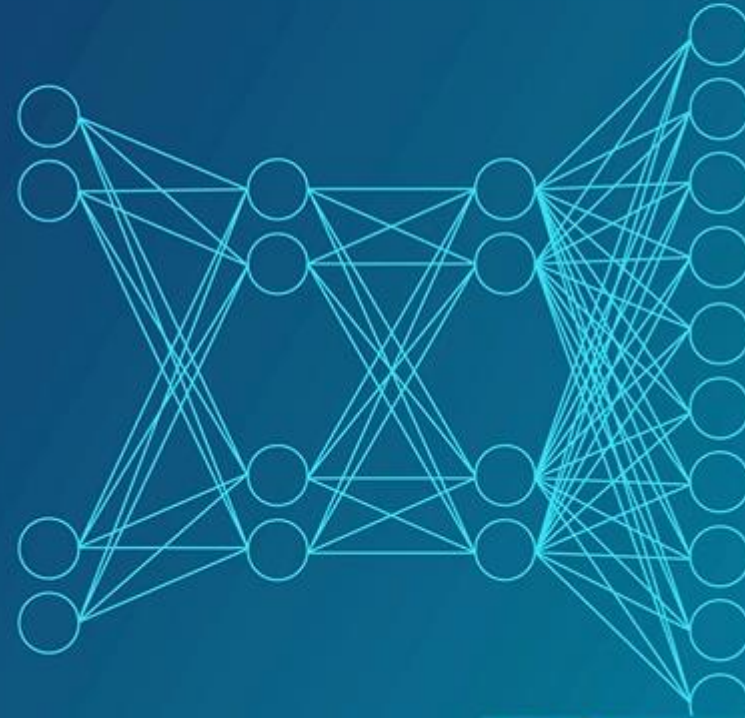
3. Linear regression with scikit-learn

Sklearn

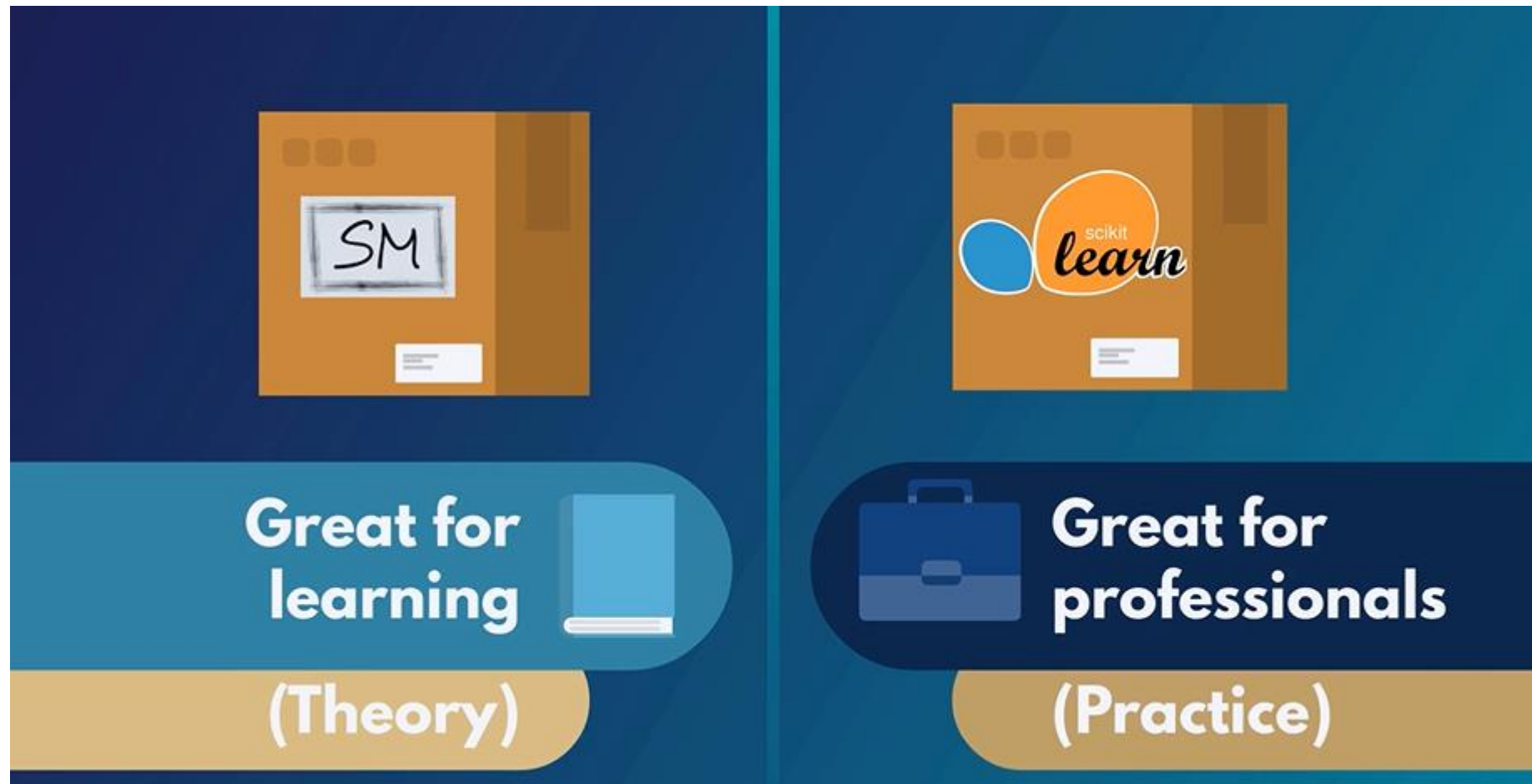
Machine learning



Deep learning



Sklearn



Example

Import the relevant libraries

```
[1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
sns.set()

from sklearn.linear_model import LinearRegression
```

Load the data

```
: data = pd.read_csv('1.01. Simple linear regression.csv')
data.head()
```

	SAT	GPA
0	1714	2.40
1	1664	2.52
2	1760	2.54
3	1685	2.74
4	1693	2.83

Declare the dependent and independent variables

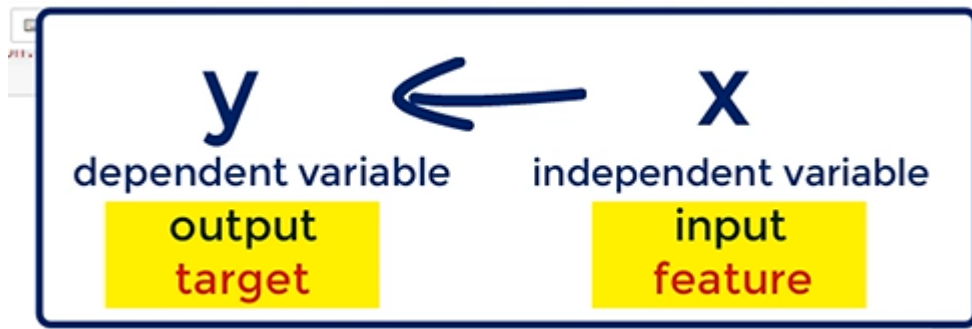
```
: x = data['SAT']
y = data['GPA']
```

```
: x_matrix = x.values.reshape(-1,1)
x_matrix.shape
```

Regression itself

```
: reg = LinearRegression()
```

```
: reg.fit(x_matrix,y)
```



Example

R-squared

```
In [10]: reg.score(x_matrix,y)
```

```
Out[10]: 0.40600391479679765
```

Coefficients

```
: reg.coef_
```

```
: array([0.00165569])
```

Intercept

```
: reg.intercept_
```

```
: 0.2750402996602803
```

StatsModels summary for the same regression

OLS Regression Results

Dep. Variable:	GPA	R-squared:	0.406
Model:	OLS	Adj. R-squared:	0.399
Method:	Least Squares	F-statistic:	50.05
Date:	Mon, 01 Apr 2019	Prob (F-statistic):	7.20e-11
Time:	16:06:24	Log-Likelihood:	12.672
No. Observations:	84	AIC:	-21.34
Df Residuals:	82	BIC:	-16.48
Df Model:	1		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
const	0.2750	0.409	0.673	0.503	-0.538	1.088
SAT	0.0017	0.000	7.487	0.000	0.001	0.002

Omnibus:	12.839	Durbin-Watson:	0.950
Prob(Omnibus):	0.002	Jarque-Bera (JB):	16.155
Skew:	-0.722	Prob(JB):	0.000310
Kurtosis:	4.590	Cond. No.	3.29e+04

Making predictions

```
: reg.predict(1740)
```

```
: array([3.15593751])
```

```
: new_data = pd.DataFrame(data=[1740,1760],columns=['SAT'])
   new_data
```

```
:
   SAT
0  1740
1  1760
```

```
: reg.predict(new_data)
```

```
: array([3.15593751, 3.18905127])
```

Example

Formula for Adjusted R²

$$R_{adj.}^2 = 1 - (1 - R^2) * \frac{n-1}{n-p-1}$$

```

: r2 = reg.score(x,y)
  n = x.shape[0]
  p = x.shape[1]

adjusted_r2 = 1-(1-r2)*(n-1)/(n-p-1)
adjusted_r2

: 0.39203134825134023

```

StatsModels summary for the same regression

OLS Regression Results

Dep. Variable:	GPA	R-squared:	0.407
Model:	OLS	Adj. R-squared:	0.392
Method:	Least Squares	F-statistic:	21.76
Date:	Thu, 14 Mar 2019	Prob (F-statistic):	6.58e-10
Time:	11:36:31	Log-Likelihood:	12.720
No. Observations:	84	AIC:	-19.44
Df Residuals:	81	BIC:	-12.15
Df Model:	2		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
const	0.2960	0.417	0.710	0.480	-0.533	1.125
SAT	0.0017	0.000	7.432	0.000	0.001	0.002
Rand 1,2,3	-0.0083	0.027	-0.304	0.762	-0.062	0.046

Omnibus:	12.992	Durbin-Watson:	0.948
Prob(Omnibus):	0.002	Jarque-Bera (JB):	16.364
Skew:	-0.731	Prob(JB):	0.000280
Kurtosis:	4.594	Cond. No.	3.33e+04

Feature selection

Feature selection simplifies models, improves speed and prevents a series of unwanted issues arising from having too many features

Feature selection

```
11]: from sklearn.feature_selection import f_regression
12]: f_regression(x,y)
12]: (array([56.04804786, 0.17558437]), array([7.19951844e-11, 6.76291372e-01]))
```

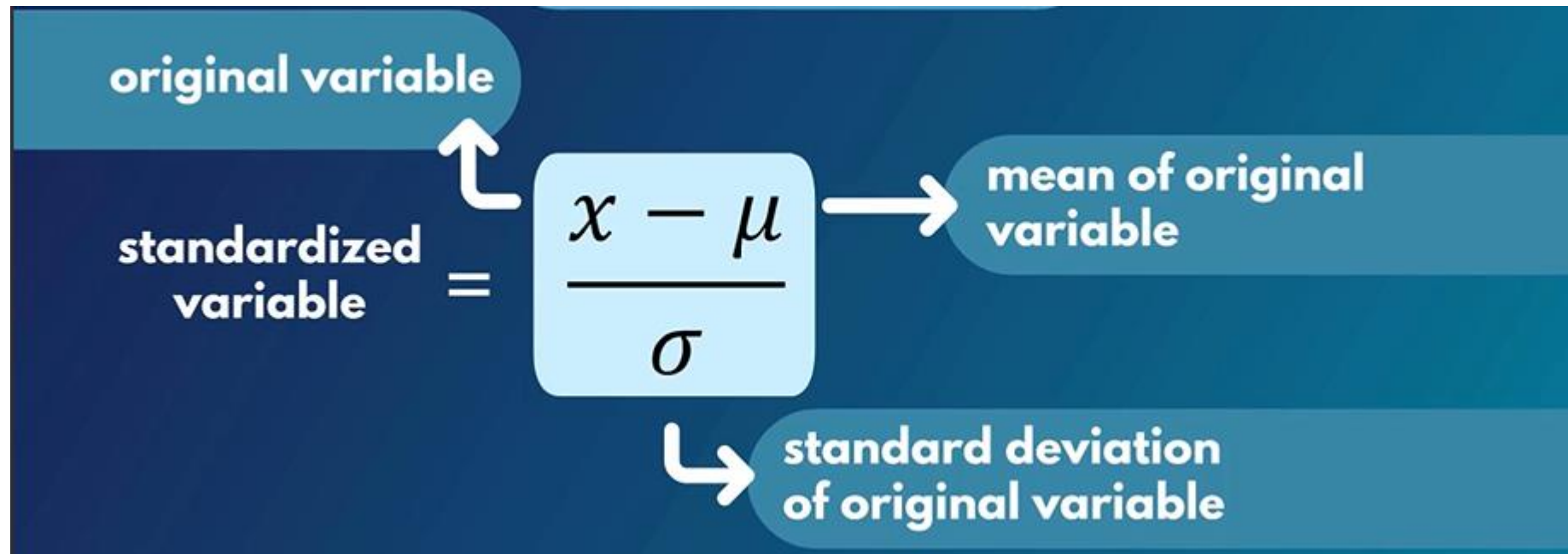
F-statistics **p-values**

Declare the dependent and independent variables

```
In [4]: x = data[['SAT', 'Rand 1,2,3']]
        y = data['GPA']
```

```
In [14]: p_values.round(3)
Out[14]: array([0. , 0.676])
```

Feature scaling



Feature scaling

Day	Exchange rate	Daily trading volume
1	1.3	110,000
2	1.34	98,700
3	1.25	135,000
	mean: 1.3	
	std: 0.045	

Exchange rate
0.07 (1.3)
0.96 (1.34)
-1.03 (1.25)

Daily trading volume
-0.25 (110,000)
-0.85 (98,700)
1.1 (135,000)

Standardization

```

from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

scaler.fit(x)

StandardScaler(copy=True, with_mean=True, with_std=True)

x_scaled = scaler.transform(x)

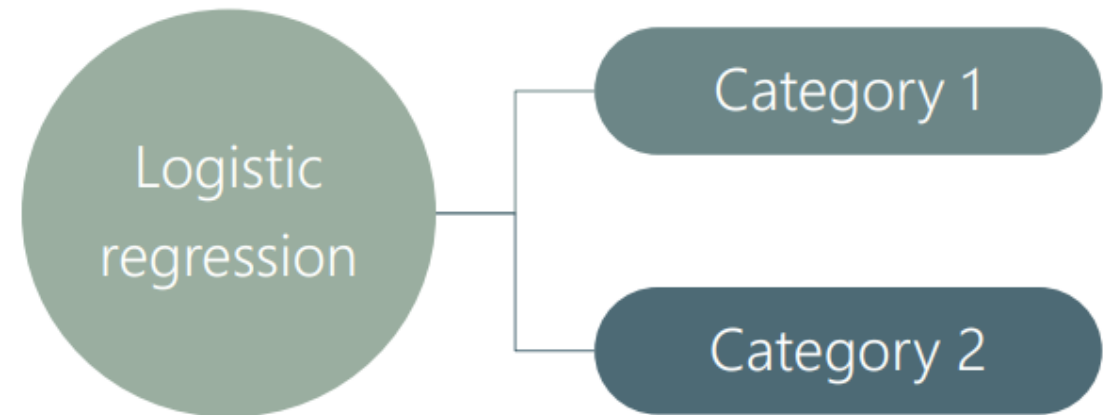
x_scaled
array([[ -1.26338288,  -1.24637147],
       [ -1.74458431,   1.10632974],
       [ -0.82067757,   1.10632974],
       [ -1.54247971,   1.10632974],
       [ -1.46548748,  -0.07002087],
       [ -1.68684014,  -1.24637147],
       [ -0.78218146,  -0.07002087],
       [ -0.78218146,  -1.24637147],
       [ -0.51270866,  -0.07002087],
       [  0.04548499,   1.10632974],
       [ -1.06127829,   1.10632974],
       [ -0.67631715,  -0.07002087],
       [ -1.06127829,  -1.24637147],

```

4.Logistic regression

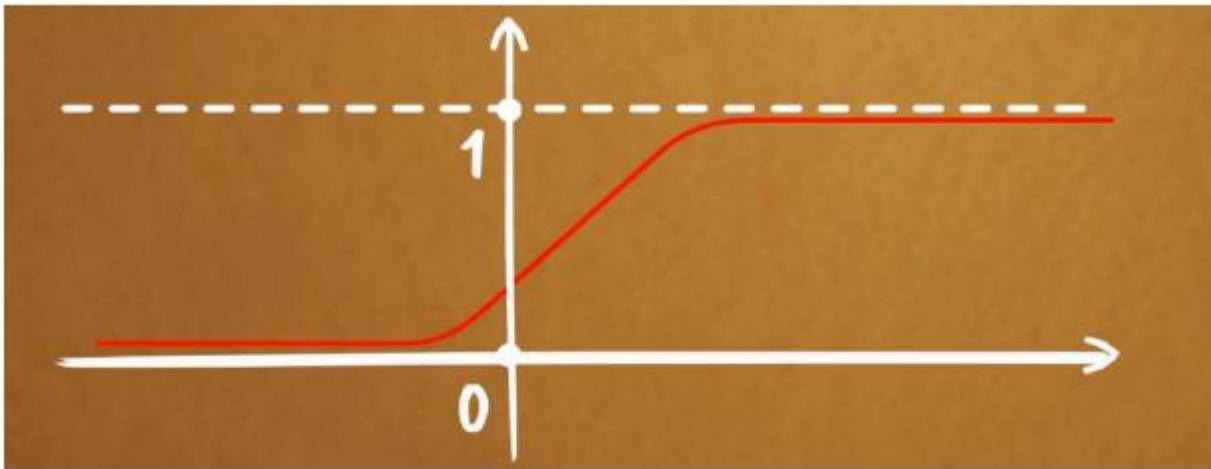
Logistic regression

- Logistic regression implies that the possible outcomes are not numerical but rather categorical.
- Examples for categories are:
 - Yes / No
 - Will buy / Won't Buy



Logistic model

- The logistic regression predicts the probability of an event occurring



Visual representation of a logistic function

Logistic regression model

- The logistic regression model is not very useful in itself. The right-hand side of the model is an exponent which is very computationally inefficient and generally hard to grasp.

$$p(X) = \frac{e^{(\beta_0 + \beta_1 X_1 + \dots + \beta_k X_k)}}{1 + e^{(\beta_0 + \beta_1 X_1 + \dots + \beta_k X_k)}}$$

$$\frac{p(X)}{1 - p(X)} = e^{(\beta_0 + \beta_1 X_1 + \dots + \beta_k X_k)}$$

$$\begin{aligned} p(X)(1 + e^{(\beta_0 + \beta_1 x_1 + \dots + \beta_k x_k)}) &= e^{(\beta_0 + \beta_1 x_1 + \dots + \beta_k x_k)} \\ p(X) * 1 + p(X)e^{(\beta_0 + \beta_1 x_1 + \dots + \beta_k x_k)} &= e^{(\beta_0 + \beta_1 x_1 + \dots + \beta_k x_k)} \\ p(X) &= e^{(\beta_0 + \beta_1 x_1 + \dots + \beta_k x_k)} - p(X)e^{(\beta_0 + \beta_1 x_1 + \dots + \beta_k x_k)} \\ p(X) &= (1 - p(X))e^{(\beta_0 + \beta_1 x_1 + \dots + \beta_k x_k)} \end{aligned}$$

Logistic regression model

- When we talk about a 'logistic regression' what we usually mean is 'logit' regression – a variation of the model where we have taken the log of both sides.

$$\log \left(\frac{p(X)}{1-p(X)} \right) = \log(e^{(\beta_0 + \beta_1 x + \dots + \beta_k x_k)})$$
$$\log \left(\frac{p(X)}{1-p(X)} \right) = \beta_0 + \beta_1 x + \dots + \beta_k x_k$$

Example

■ GPA -> Admittance

```
import numpy as np
import pandas as pd
import statsmodels.api as sm
import matplotlib.pyplot as plt
import seaborn as sns
sns.set()
```

```
y = data['Admitted']
x1 = data['SAT']
```

```
raw_data = pd.read_csv('2.01. Admittance.csv')
raw_data
```

```
data = raw_data.copy()
data['Admitted'] = data['Admitted'].map({'Yes':1, 'No':0})
data
```

```
x = sm.add_constant(x1)
```

```
reg_log = sm.Logit(y,x)
results_log = reg_log.fit()
```

Understanding Logistic Regression Table



Regression summary

Logit Regression Results

Dep. Variable:	Admitted	No. Observations:	168
Model:	Logit	Df Residuals:	166
Method:	MLE	Df Model:	1
Date:	Thu, 17 May 2018	Pseudo R-squ.:	0.7992
Time:	15:51:29	Log-Likelihood:	-23.145
converged:	True	LL-Null:	-115.26
		LLR p-value:	5.805e-42

	coef	std err	z	P> z	[0.025	0.975]
const	-69.9128	15.737	-4.443	0.000	-100.756	-39.070
SAT	0.0420	0.009	4.454	0.000	0.024	0.060

Maximum likelihood estimation (MLE)

Likelihood function:

a function which estimates how likely it is that the model at hand describes the real underlying relationship of the variables.

The bigger the likelihood function, the higher the probability that our model is correct!

MLE tries to maximize the likelihood function.

The computer is going through different values, until it finds a model, for which the likelihood is the highest.

When it can no longer improve it, it will just stop the optimization.

Understanding Logistic Regression Table



Regression summary

Logit Regression Results

Dep. Variable:	Admitted	No. Observations:	168
Model:	Logit	Df Residuals:	166
Method:	MLE	Df Model:	1
Date:	Thu, 17 May 2018	Pseudo R-squ.:	0.7992
Time:	15:51:29	Log-Likelihood:	-23.145
converged:	True	LL-Null:	-115.26
		LLR p-value:	5.805e-42

	coef	std err	z	P> z	[0.025	0.975]
const	-69.9128	15.737	-4.443	0.000	-100.756	-39.070
SAT	0.0420	0.009	4.454	0.000	0.024	0.060

Pseudo R-squared:

McFadden's R-squared

'A good Pseudo R-squared is somewhere between 0.2 and 0.4.

This measure is mostly useful for comparing variations of the same model.

Different models will have completely different and incomparable Pseudo R-squares!'

Understanding Logistic Regression Table



Regression summary

Logit Regression Results

Dep. Variable:	Admitted	No. Observations:	168			
Model:	Logit	Df Residuals:	166			
Method:	MLE	Df Model:	1			
Date:	Thu, 17 May 2018	Pseudo R-squ.:	0.7992			
Time:	15:51:29	Log-Likelihood:	-23.145			
converged:	True	LL-Null:	-115.26			
		LLR p-value:	5.805e-42			
	coef	std err	z	P> z	[0.025	0.975]
const	-69.9128	15.737	-4.443	0.000	-100.756	-39.070
SAT	0.0420	0.009	4.454	0.000	0.024	0.060

Logit model

SAT - Admittance regression

$$\log\left(\frac{\pi}{1-\pi}\right) = -69.91 + 0.042 * SAT$$

π -> probability of an event occurring

$1 - \pi$ -> probability of the event NOT occurring

Log-Likelihood Ratio p-value
measures of our model is statistically
different from the benchmark 'worst'
model.

Caculating the Accuracy of the model

```
np.set_printoptions(formatter={'float': lambda x: "{0:0.2f}".format(x)})
results_log.predict()
```

```
array([0.00, 1.00, 1.00, 0.23, 0.02, 0.99, 1.00, 1.00, 1.00, 0.01, 1.00,
       1.00, 0.76, 0.00, 0.60, 1.00, 0.11, 0.12, 0.51, 1.00, 1.00, 1.00,
       0.00, 0.01, 0.97, 1.00, 0.48, 0.99, 1.00, 0.99, 0.00, 0.83, 0.25,
       1.00, 1.00, 1.00, 0.31, 1.00, 0.23, 0.00, 0.02, 0.45, 1.00, 0.00,
       0.99, 0.00, 0.99, 0.00, 0.00, 0.01, 0.00, 1.00, 0.92, 0.02, 1.00,
       0.00, 0.37, 0.98, 0.12, 1.00, 0.00, 0.78, 1.00, 1.00, 0.98, 0.00,
       0.00, 0.00, 1.00, 0.00, 0.78, 0.12, 0.00, 0.99, 1.00, 1.00, 0.00,
       0.30, 1.00, 1.00, 0.00, 1.00, 1.00, 0.85, 1.00, 1.00, 0.00, 1.00,
       1.00, 0.89, 0.83, 0.00, 0.98, 0.97, 0.00, 1.00, 1.00, 0.03, 0.99,
       0.96, 1.00, 0.00, 1.00, 0.01, 0.01, 1.00, 1.00, 1.00, 0.00, 0.00,
       0.02, 0.33, 0.00, 1.00, 0.09, 0.00, 0.97, 0.00, 0.75, 1.00, 1.00,
       0.01, 0.01, 0.00, 1.00, 0.00, 0.99, 0.57, 0.54, 0.87, 0.83, 0.00,
       1.00, 0.00, 0.00, 0.00, 1.00, 0.04, 0.00, 0.01, 1.00, 0.99, 0.52,
       1.00, 1.00, 0.05, 0.00, 0.00, 0.00, 0.68, 1.00, 1.00, 1.00, 1.00,
       1.00, 0.00, 1.00, 1.00, 0.04, 1.00, 0.02, 1.00, 0.99, 0.97, 0.94,
       0.01, 0.00, 0.00])
```

**PREDICTED VALUES
BY THE MODEL**

```
np.array(data['Admitted'])
```

```
array([0, 1, 1, 0, 0, 1, 1, 1, 1, 0, 1, 1, 1, 0, 0, 1, 0, 0, 1, 1, 1, 1,
       0, 0, 1, 1, 1, 1, 1, 1, 0, 1, 0, 1, 1, 1, 0, 1, 0, 0, 0, 1, 1, 0,
       1, 0, 1, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 1, 0, 1, 0, 1, 1, 1, 1, 0,
       0, 0, 1, 0, 1, 1, 0, 1, 1, 1, 0, 1, 1, 1, 0, 1, 1, 0, 1, 1, 0, 1,
       1, 1, 0, 0, 1, 1, 0, 1, 1, 0, 1, 1, 1, 0, 1, 0, 0, 1, 1, 1, 0, 0,
       0, 0, 1, 0, 0, 1, 0, 1, 1, 1, 0, 0, 0, 1, 0, 1, 0, 1, 1, 1, 0,
       1, 0, 0, 0, 1, 0, 0, 0, 1, 1, 1, 1, 1, 0, 0, 0, 0, 1, 1, 1, 1, 1,
       1, 0, 1, 1, 0, 1, 0, 1, 1, 1, 1, 0, 0, 0, 0], dtype=int64)
```

ACTUAL VALUES

```
In [11]: results_log.pred_table()
```

```
Out[11]: array([[69.00, 5.00],
               [4.00, 90.00]])
```

```
In [12]: cm_df = pd.DataFrame(results_log.pred_table())
cm_df.columns = ['Predicted 0', 'Predicted 1']
cm_df = cm_df.rename(index={0: 'Actual 0', 1: 'Actual 1'})
cm_df
```

```
Out[12]:
```

	Predicted 0	Predicted 1
Actual 0	69.0	5.0
Actual 1	4.0	90.0

CONFUSION MATRIX

Tổng kết

- Phân tích hồi quy là một trong những phương pháp được sử dụng rộng rãi nhất để dự đoán.
- Các giả định để sử dụng được mô hình hồi qui
- Một số vấn đề về xử lý dữ liệu categorical, scaling